

AgenQA: An Agentic Harness for Synthesizing Challenging Reasoning QA via Growth and Folding of Step-Verifiable Dependency Chains

冯亮

Research project preview

2026 年 5 月

摘要

AgenQA 研究如何合成具有挑战性的 scientific reasoning QA 数据，同时不失去可验证性。它先构造一条 step-verifiable Chain-of-KQA，再通过 Path-Fold 隐藏中间事实，形成更难 solver-facing Path View；局部 Edge View 则保留 step-level correctness control 所需的可见支撑。系统上，AgenQA 将生成过程组织为 Director–Operator–Evaluator loop，使 generation、verification 和 repair 发生在同一显式 reasoning state 上。阶段性评测显示，AgenQA 生成的 Path View 题目能形成有效的 model-discrimination signal: SOTA solver 评测整体准确率为 84.18%，诊断子集准确率为 51.77%；Qwen-family 梯度评测随模型规模从 48.00% 提升到 66.86%。

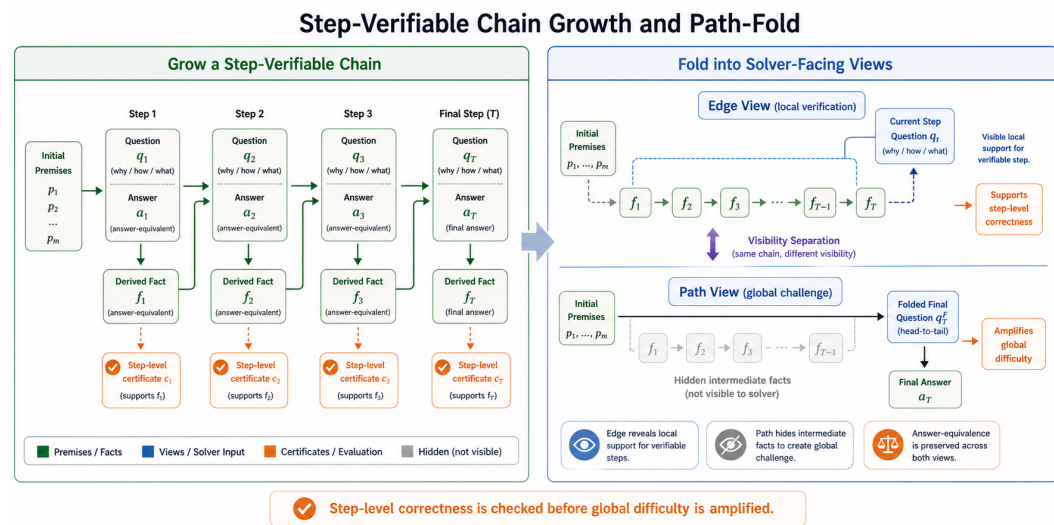


图 1: AgenQA overview: grow a step-verifiable chain, then fold it into Edge/Path solver-facing views.

1 Motivation and Core Idea

面向 challenging reasoning QA 的高质量数据不只服务于 benchmark，它同时是评估和改进大语言模型的共同数据基底。随着模型具备更强的指令遵循能力和更长的 inference-time reasoning 能力，扁平 question-answer data 的诊断价值正在下降：最终答案只能告诉我们模型是否答对，却很难解释成功来自稳健的多步推理、浅层模式匹配、幸运猜测，还是训练阶段已经见过相似实例。

问题不只是“生成更多题目”。MegaScience [1] 和 Data Darwinism [2] 等工作都说明，高价值数据需要可靠性、可检查性、领域覆盖和可复用的数据对象。已有 synthetic reasoning-data 方法已经从 one-shot text generation 走向受控构造过程：evolution-based methods 改写或提升已有题目 [3, 4, 5, 6]，composition-based methods 放大 long-horizon structure [7, 8]，CHASE 这类 bottom-up methods 从可验证子任务构造困难题 [9]，BenchAgents 和 Agentic Proposing 则进一步引入 agentic planning、verification 或 sequential decision making [10, 11]。

这些方向共同暴露了一个核心张力：difficulty 与 correctness 往往被耦合在一起。直接生成最终难题容易产生表面复杂但不可验证的问题；把题目拆成完全暴露的小步骤又会削弱全局难度。AgenQA 的判断是，hard-QA synthesis 需要一个显式 control object：从初始 known context 连接到最终答案的 dependency path。生成侧保留这条 path 用于检查、修复和审计；solver-facing 侧通过 Path-Fold 隐藏中间结论，使难度来自对可靠路径的重构，而不是不可验证的一次性复杂化。

AgenQA 因此围绕 step-level correctness control 与 global difficulty amplification 的分离来组织 synthesis。它先 progressive construction 一条 step-verifiable **Chain-of-KQA**，再通过 **Path-Fold** 隐藏中间推导步骤，把同一条 chain 投影为更难的 **Path View**；同时保留局部 **Edge View**，用于验证单步 transition 是否 well-posed、answer-equivalent 且可被强 solver 稳定求解。

2 Contributions

AgenQA 的贡献可以概括为三点：

1. **Edge/Path-grounded Chain-of-KQA formalism.** AgenQA 将 challenging reasoning-QA synthesis 表示为一条逐步增长的 step-verifiable dependency chain。每个局部 transition 都带有可检查的 known state、answer-equivalent key fact 和 dependency certificate；同一条 chain 再被投影为 Edge View 与 Path View，从而把 step-level correctness control 和 global difficulty amplification 明确拆开。
2. **Agentic state-transition harness for synthesis and repair.** AgenQA 不把 agent 当成一次性出题器，而是把 Director、Operators 和 Evaluator 组织成显式 reasoning state 上的闭环。Extension、revision、folding、solver feedback、consensus 和 acceptance signals 都作用在同一 Chain-of-KQA object 上，使 generation、verification 和 repair 成为一个可追踪生命周期。

3. **Benchmark-construction validation with model-discrimination signals.** 当前 public preview 以 benchmark construction 为主验证面: Edge-side solver signals 用作 correctness 与 well-posedness gates, Path-side solver distributions 用作 difficulty 与 discrimination signals。阶段性结果显示, Path View items 能区分 SOTA solvers 与不同规模 Qwen-family models, 并保留可解释的难度来源。

3 AgenQA Framework

AgenQA 不把 challenging reasoning-QA synthesis 视为一次性生成最终难题, 而是把它实现为显式 reasoning state 上的 growth、folding、evaluation 和 repair。

3.1 Chain-of-KQA and Path-Fold

普通 QA 可以写成

$$x = (K, q, a),$$

其中 K 是给 solver 的已知条件, q 是问题, a 是目标答案。这个表示只描述最终题目, 却没有说明题目如何被构造出来。AgenQA 改为 progressive generation: 每一步只在当前已知状态上扩展一个局部可检查、与答案等价的新 fact。一个步骤是一个 KQA transition:

$$s_t = (K_t, q_t, a_t, c_t),$$

其中 K_t 是第 t 步前可见的支撑信息, q_t 是该步局部问题, a_t 是该步得到的答案等价结论, c_t 是 dependency certificate。

Certificate 记录这一答案从哪里来:

$$c_t = (U_t, \Delta F_t, f_t),$$

其中 U_t 是该步使用的前提或先前事实, ΔF_t 是该步新产生的事实, f_t 是与 a_t 等价的关键事实。这些关键事实形成一条 dependency spine:

$$K_0 \rightarrow f_1 \rightarrow f_2 \rightarrow \cdots \rightarrow f_T.$$

这可以避免生成结果退化成一组松散相关的问题。每条 accepted chain 同时具有 locally auditable transitions 和 global dependency direction。

AgenQA 将同一条 underlying chain 暴露为两种 solver-facing views。Edge View 暴露求解当前局部问题所需的支撑信息; Edge success 因此可以作为 step-level correctness 和 well-posedness 的代理信号。Path View 构造 chain-level challenge: Path-Fold 保留原始前提、定义和必要条件, 但隐藏中间 key facts 和 step certificates。Solver 只能看到折叠后的最终问题, 需从可见上下文恢复通向最终答案的 latent dependency path。

一个有效 fold 至少满足:

- **Answer equivalence:** 折叠后的问题仍然以尾部步骤的答案为目标答案。

- **Visibility separation:** 中间 key facts、step history 和内部指针不能泄露给 solver。
- **Path preservation:** 题目仍然可以求解，但需要重构 dependency path 或等价推理路径。

3.2 Agentic State-Transition Harness

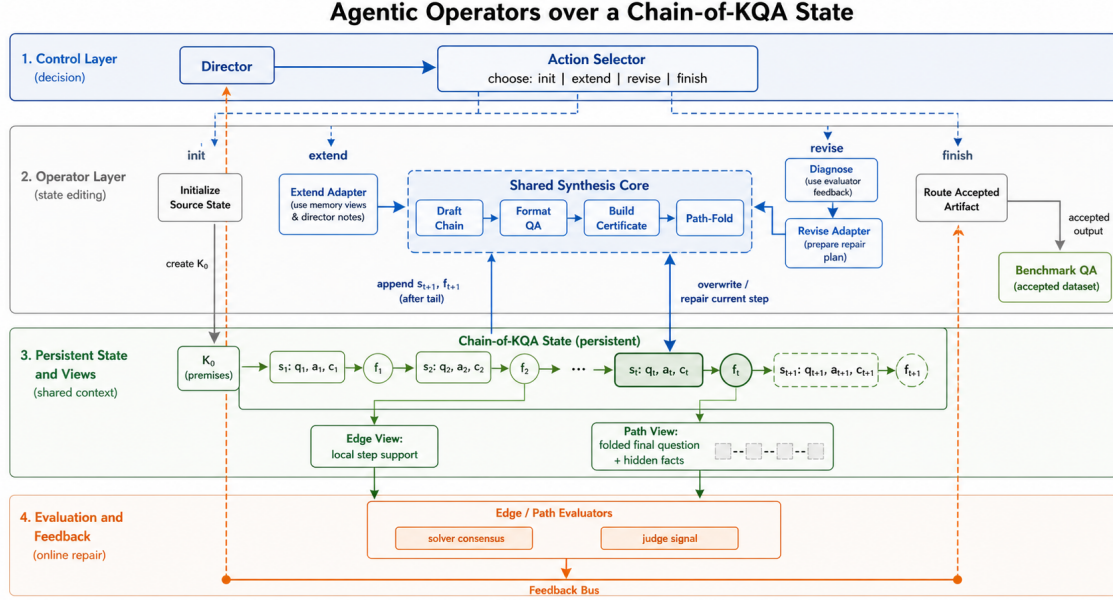


图 2: Agentic operators over a Chain-of-KQA state. Extension and revision reuse the same synthesis core but apply different state-edit semantics.

整个 synthesis process 是作用在 progressive chain 及其 solver-facing views 上的 controlled state-transition loop:

$$d_r = D(S_r), \quad S_{r+1} = O_{d_r}(S_r), \quad z_{r+1} = E(\Pi(S_{r+1})).$$

其中 S_r 是第 r 轮 synthesis state, D 是 Director controller, d_r 是被选择的 operation, O 是对应 Operator, Π 构造 Edge/Path projections, E 返回 solver、consensus 和 acceptance signals。

Director. Director 负责从 available operator set 中选择下一步。它读取 progress limits、recent chain history、memory windows、available operations、question-type policy、Edge/Path solver outcomes、consensus summaries、ambiguity / contract reports 和 repair history。

Operators. Operators 是作用在 Chain-of-KQA state 上的 structured generation procedures。Initialization 构造 starting anchor。Extension 追加新的 KQA transition。Revision 修复最新的问题 transition，同时保持该步骤在 chain 中的 intended role。在 semantic track 中，extension 和 revision 共享同一条核心 role sequence:

$$\text{draft} \rightarrow \text{format} \rightarrow \text{certify} \rightarrow \text{fold}.$$

二者区别在于 adapter logic 和 state-edit semantics: extension 追加新步骤, revision 覆盖或修复有问题的步骤。

Evaluator. Evaluator 检查生成出的 Edge 和 Path views。Solvers 同时回答两种 views, consensus 聚合多个 strong-solver judgments, acceptance logic 决定当前 chain 是继续 extend、revise、accept 还是 terminate。Evaluator feedback 不是 post-hoc filter, 而是 online state 的一部分, 会影响后续 Director decisions。

3.3 Scalability and Extensibility

这个 harness 可扩展, 是因为增加 chain length 或 task difficulty 不需要重新设计一个 final-question generator。每一轮都应用同一个 state-transition interface: extension 追加 transition, revision 修复问题 transition, evaluation 读取 Edge/Path projections, routing 导出 accepted states。

这个 harness 可扩展, 也是因为 formal object 与 domain-specific implementations 分离。Source grounding、operator adapters、view constructors、solver ensembles 和 acceptance policies 都可以替换, 同时保留 Chain-of-KQA、Edge View 和 Path View 这套接口。

4 Experimental Showcase

4.1 Evaluation Results

本节保留两张最能说明问题的表: 一张展示强模型在 Path View 题目上的表现, 另一张展示 Qwen 系列随模型规模变化的能力梯度。两张表共同说明: AgenQA 的 Path-Fold 不只是产生更长题面, 而是在保留生成侧 dependency state 的同时, 为 solver-facing benchmark 构造出可测的 difficulty 与 model-discrimination signal。

表 1: SOTA solvers on Path View questions.

Solver	All Path View	Accuracy	Diagnostic	Diagnostic Acc.
claude-sonnet-4-6	466 / 574	81.18%	63 / 150	42.00%
gemini-3.1-pro-preview	254 / 294	86.39%	47 / 78	60.26%
glm-5	494 / 574	86.06%	89 / 150	59.33%
gpt-5.4-2026-03-05	469 / 574	81.71%	62 / 150	41.33%
qwen3.5-plus	498 / 575	86.61%	90 / 150	60.00%
All solvers	2181 / 2591	84.18%	351 / 678	51.77%

表 2: Qwen-family scale gradient on Path View questions.

Solver	All Path View Acc.	Diagnostic Acc.	Interpretation
qwen3-4b	48.00%	21.55%	lower-capacity baseline
qwen3-8b	56.57%	34.48%	mid-scale improvement
Qwen3-32B	66.86%	50.00%	larger-model improvement
All solvers	59.54%	38.97%	family-level aggregate

4.2 Representative Path View Samples

下面展示三道 selected Path View sample questions。它们都属于 solver-facing Path View：题面只给出最终求解所需的可见条件，中间 dependency path 被折叠，解题者需要自行恢复关键推理步骤。

Sample 1: FedAvg 收敛界中的学习率选择

Subject: Federated and Distributed Learning.

Question. 给定 FedAvg 设定：

- 全局目标函数为

$$F(w) = \sum_k \frac{n_k}{n} F_k(w),$$

其中 $F_k(w)$ 是客户端 k 的局部经验损失， $n_k = |D_k|$ ，且 $n = \sum_k n_k$ 。

- 每个客户端从 w^t 出发，以常数学习率 $\eta > 0$ 执行 $E > 1$ 步本地 SGD，再由服务器按权重平均聚合。
- 客户端数据 non-IID，梯度异质性由

$$G = \left(\sum_k \frac{n_k}{n} \|\nabla F_k(w) - \nabla F(w)\|^2 \right)^{1/2}$$

表示。全局损失满足 L -smoothness，一共进行 T 轮通信。

要求求出使时间平均平方梯度范数上界最小的学习率 η^* ，并给出优化后的最紧上界。

Reference answer.

$$\eta^* = \sqrt{\frac{2[F(w^0) - F(w^*)]}{LE^2G^2T}}, \quad \frac{1}{T} \sum_{t=0}^{T-1} \|\nabla F(w^t)\|^2 \leq 2\sqrt{\frac{2LG^2[F(w^0) - F(w^*)]}{T}}.$$

Sample 2: Pointer-Generator Coverage Loss 的梯度路径

Subject: Natural Language Processing / Abstractive Summarization.

Question. 给定软注意力 encoder-decoder：

$$e_{t,i} = \text{score}(s_t, h_i), \quad a_{t,i} = \frac{\exp(e_{t,i})}{\sum_j \exp(e_{t,j})}, \quad c_t = \sum_i a_{t,i} h_i.$$

组合训练损失为

$$L_{\text{total}} = L_{\text{NLL}} + \lambda L_{\text{cov}}, \quad \text{cov}_{t,i} = \sum_{t' < t} a_{t',i}, \quad L_{\text{cov}} = \sum_t \sum_i \min(a_{t,i}, \text{cov}_{t,i}).$$

要求推导 $\partial L_{\text{total}} / \partial h_i$ 的单一闭式符号表达, 同时捕捉 score path 与 context path。

Reference answer.

$$\frac{\partial L_{\text{total}}}{\partial h_i} = \sum_t \left[a_{t,i} \left(g_{t,i} - \sum_j a_{t,j} g_{t,j} \right) \frac{\partial e_{t,i}}{\partial h_i} + a_{t,i} \frac{\partial L_{\text{NLL}}}{\partial c_t} \right].$$

Sample 3: TRADES 鲁棒训练目标的外层与内层条件

Subject: Machine Learning Security / Adversarial Robustness.

Question. 给定参数化深度神经网络 f_θ , 可行扰动集合为闭 L_p 球 $\{\delta : \|\delta\|_p \leq \varepsilon\}$ 。TRADES-style 目标为

$$\min_{\theta} \mathbb{E}_{(x,y) \sim \mu} \left[L(f_\theta(x), y) + \beta \max_{\|\delta\|_p \leq \varepsilon} \text{KL}(f_\theta(x) \| f_\theta(x + \delta)) \right].$$

要求合并推导外层 envelope-theorem gradient、 θ^* 处外层驻点条件、内层归一化 PGD 更新规则, 以及收敛时的 KKT 条件。

Reference answer.

$$\nabla_{\theta} \max_{\|\delta\|_p \leq \varepsilon} \text{KL}(f_\theta(x) \| f_\theta(x + \delta)) = \nabla_{\theta} \text{KL}(f_\theta(x) \| f_\theta(x + \delta^*(\theta))).$$

$$\nabla_{\theta} \mathbb{E}_{(x,y) \sim \mu} [L(f_{\theta^*}(x), y)] + \beta \nabla_{\theta} \mathbb{E}_{(x,y) \sim \mu} [\text{KL}(f_{\theta^*}(x) \| f_{\theta^*}(x + \delta^*(\theta^*)))] = 0.$$

$$\delta_{t+1} = \Pi_{\|\cdot\|_p \leq \varepsilon} \left(\delta_t + \alpha \frac{\nabla_{\delta} \text{KL}(f_\theta(x) \| f_\theta(x + \delta_t))}{\|\nabla_{\delta} \text{KL}(f_\theta(x) \| f_\theta(x + \delta_t))\|_p} \right),$$

$$\nabla_{\delta} \text{KL}(f_\theta(x) \| f_\theta(x + \delta^*)) = \lambda \nabla_{\delta} \|\delta^*\|_p, \quad \lambda(\|\delta^*\|_p - \varepsilon) = 0, \quad \|\delta^*\|_p \leq \varepsilon.$$

5 Conclusion

AgenQA 的核心判断是: 高质量 challenging reasoning QA 不能只依赖 final question generation。难度与正确性需要被拆开处理: 生成侧保留可审计、可修复的 dependency path; solver 侧通过 Path-Fold 面对被折叠后的端到端问题。这样, 系统可以在每一步维持 step-level verifiability, 同时让最终问题承载更长的依赖路径和更强的模型区分度。

从工程上看, Director–Operator–Evaluator loop 使 AgenQA 不只是一个 prompt pipeline, 而是一个围绕显式 reasoning state 运转的 synthesis harness。它能够把 extension、revision、solver feedback、consensus、contract stabilization 和 artifact replay 组织进同一生命周期。当前展示的评测表格与样例题说明, 这一设计已经能产生有可读形态、有难度信号、也能解释其难度来源的 Path View items。

参考文献

- [1] Run-Ze Fan, Zengzhi Wang, and Pengfei Liu. MegaScience: Pushing the Frontiers of Post-Training Datasets for Science Reasoning. arXiv:2507.16812, 2025.
- [2] Yiwei Qin, Zhen Huang, Tiantian Mi, Weiye Si, Chenyang Zhou, Qipeng Guo, Siyuan Feng, and Pengfei Liu. Data Darwinism Part I: Unlocking the Value of Scientific Data for Pre-training. arXiv:2602.07824, 2026.
- [3] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-Instruct: Aligning Language Models with Self-Generated Instructions. arXiv:2212.10560, 2023.
- [4] Haipeng Luo et al. WizardMath: Empowering Mathematical Reasoning for Large Language Models via Reinforced Evol-Instruct. arXiv:2308.09583, 2025.
- [5] Longhui Yu et al. MetaMath: Bootstrap Your Own Mathematical Questions for Large Language Models. arXiv:2309.12284, 2024.
- [6] Weihao Zeng, Can Xu, Yingxiu Zhao, Jian-Guang Lou, and Weizhu Chen. Automatic Instruction Evolving for Large Language Models. arXiv:2406.00770, 2024.
- [7] Yi Lu et al. R-Horizon: How Far Can Your Large Reasoning Model Really Go in Breadth and Depth? arXiv:2510.08189, 2025.
- [8] Xin Xu et al. Composition-RL: Compose Your Verifiable Prompts for Reinforcement Learning of Large Language Models. arXiv:2602.12036, 2026.
- [9] Arkil Patel, Siva Reddy, and Dzmitry Bahdanau. How to Get Your LLM to Generate Challenging Problems for Evaluation. arXiv:2502.14678, 2025.
- [10] Natasha Butt, Varun Chandrasekaran, Neel Joshi, Besmira Nushi, and Vidhisha Balachandran. BenchAgents: Multi-Agent Systems for Structured Benchmark Creation. arXiv:2410.22584, 2025.
- [11] Zhengbo Jiao et al. Agentic Proposing: Enhancing Large Language Model Reasoning via Compositional Skill Synthesis. arXiv:2602.03279, 2026.